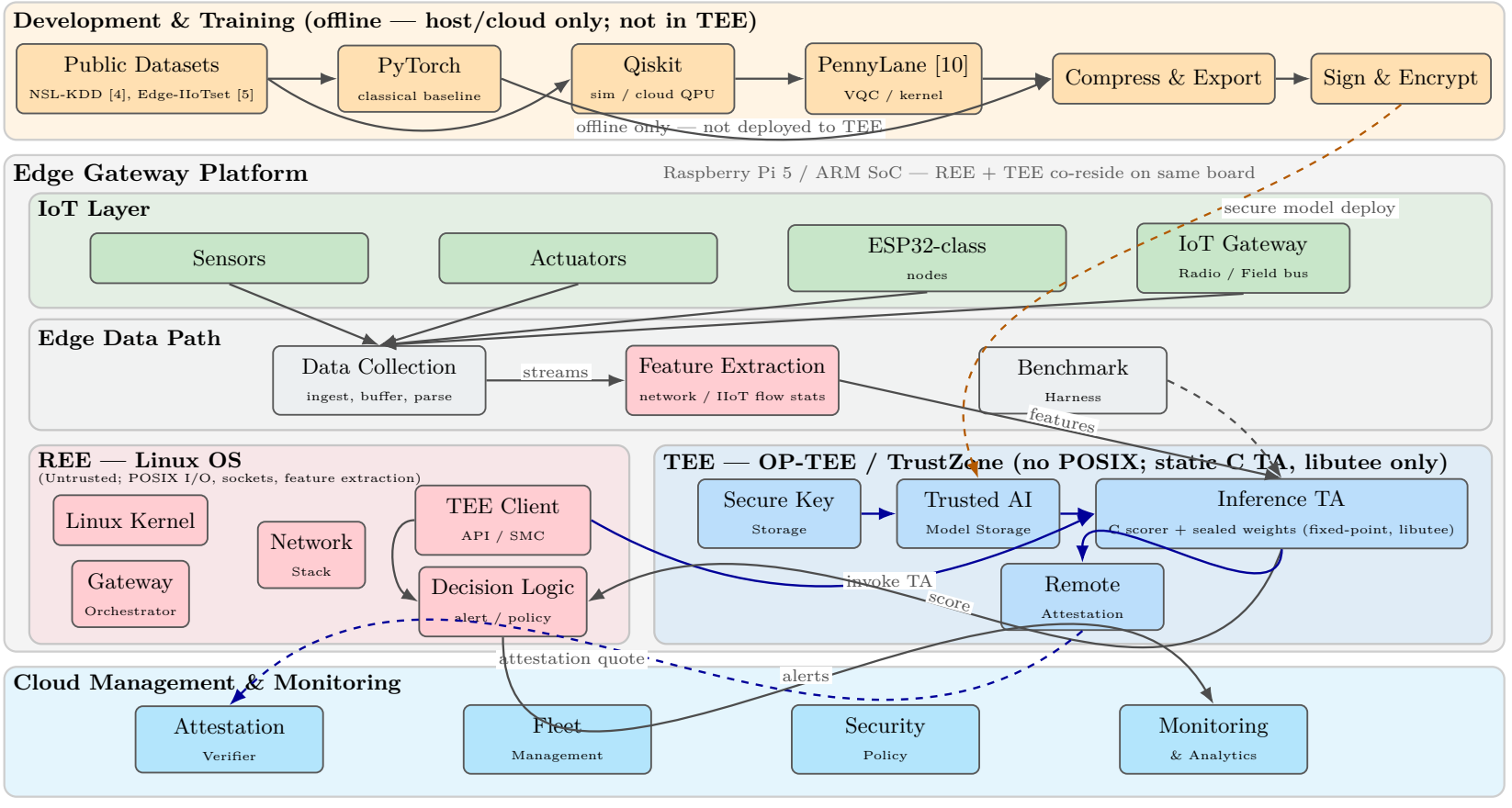
Trustworthy Hybrid Quantum–Classical AI for Secure IoT Edge Systems

Reference architecture (TRL 2–3) — Raspberry Pi 5 / ARM gateway; REE handles I/O, OP-TEE TA runs protected C inference only



Implementation Detail — Reviewer Q&A

Companion to system architecture figure (Page 1). Platform: Raspberry Pi 5 + OP-TEE [6–8], TRL 2–3. See Page 4 for full reference list.

1. Tell the implementation detail of the current system architecture

The architecture is a **three-phase, split-world** design:

Phase A — Offline (host/cloud, not on device): Public datasets (NSL-KDD [4], Edge-IIoTset [5], Bot-IoT) are used to train a PyTorch/TinyML classical baseline [1][2] and hybrid QML models (Qiskit/PennyLane [10] on simulator or cloud QPU; IoT QML prior art [9][11]). Models are compressed and exported as a **quantum-inspired C surrogate**, then signed and encrypted for deployment.

Phase B — Online REE (Linux on Pi 5): A Client Application (CA) using `libteec` performs all POSIX I/O: network capture/ingest, parsing, feature extraction, benchmark logging, and policy actions (alert/block/log). The Gateway Orchestrator coordinates the data path; the TEE Client API opens a session to the secure-world TA.

Phase B — Online TEE (OP-TEE Trusted Applications): A single **Inference TA** (static C, fixed-point, `libutee` only) loads sealed model weights from Trusted Model Storage, runs scoring, and returns an anomaly score via `TEE_InvokeCommand`. Separate logical TAs for Secure Key Storage and Remote Attestation may be merged in early prototypes. **Qiskit/PennyLane never run inside TrustZone.**

Phase C — Cloud: Attestation Verifier checks quotes; Fleet Management and Security Policy enforce device trust; Monitoring receives alerts.

Invoke sequence (CA → TA): `TEEC_InitializeContext` → `TEEC_OpenSession` → `TEEC_InvokeCommand(INFER, feature_vector[N])` → score → `TEEC_CloseSession`. Feature vector size N is fixed at build time (dataset-specific; e.g. 30–80 floats for NSL-KDD/Edge-IIoTset flows).

Prototype status: Architecture defined (TRL 2–3); Pi 5 + OP-TEE passthrough TA and classical C inference TA **in progress**; hybrid surrogate TA is a Year 2 deliverable. All latency/CPU/RAM figures in Q5 are **engineering targets** pending benchmark validation.

2. Do you assume the QAI works as Agent AI for IDS/IPS?

No. QAI is **not** an Agent AI and not an autonomous IDS/IPS agent.

It is a **trusted anomaly-scoring function** inside one Inference TA: input = fixed feature vector, output = numeric score (+ optional attestation evidence).

IDS-like: yes — the REE Decision Logic converts scores into alerts sent to cloud monitoring.
IPS-like: only if REE policy explicitly blocks or quarantines traffic based on the score — this enforcement runs in Linux (REE), not inside the TA.
There is no autonomous planning, tool use, LLM reasoning, or self-directed action loop in the secure world.

3. What data is used by the QAI (detection engine)?

The detection engine receives a **pre-computed feature vector** produced in the REE — not raw packets and not host audit logs.

Data sources align with public IoT security benchmark datasets:

- **NSL-KDD [4] / Bot-IoT:** network flow statistics (duration, protocol, bytes, flags, service type).
- **Edge-IIoTset [5]:** IIoT gateway telemetry — packet/header fields, protocol anomalies, sensor/gateway metadata aggregated into flow windows.

The REE Data Collection and Feature Extraction stages parse telemetry and normalize values into a fixed-size vector passed to the TA.

Security features by pipeline (cost/latency in Q5)

Security property	TinyML (REE)	TinyML-in-TA	Hybrid-in-TA
Model storage	REE filesystem (plain)	TEE sealed store [6]	Same as TinyML-in-TA
Model integrity	Not protected	Signed + sealed weights	Same as TinyML-in-TA
Weight / IP confidentiality	Readable by REE processes	TZDRAM; not exposed to REE	Same as TinyML-in-TA
Inference execution	Untrusted Linux userspace	Isolated OP-TEE TA [7][8]	Same as TinyML-in-TA
Secure deployment	Plain file copy	Sign & encrypt before load	Same as TinyML-in-TA
Remote attestation	Not available	TA quote to cloud verifier	Same as TinyML-in-TA
Score provenance	Untrusted REE output	TA score via `InvokeCommand`	Same as TinyML-in-TA
Policy / alert actions	REE Decision Logic	REE Decision Logic	REE Decision Logic
Runtime tamper resistance	Low (replace `.tflite`)	High (static C TA)	Same as TinyML-in-TA
Detection engine	Classical TinyML scorer	Classical scorer in TA	Hybrid Q–C surrogate [9]

Note: TinyML-in-TA and Hybrid-in-TA share the same TEE security boundary [6–8]; Hybrid-in-TA differs only in *what* is sealed (quantum-inspired surrogate weights), not *how* it is protected. Feature vectors are still prepared in the untrusted REE before TA invoke.

4. File access, system calls, network traffic — which apply?

Data source	Used?	Role in current architecture
Network traffic	**Yes (primary)**	Captured/parsed in REE; flow stats fed to feature extractor and then to TA.
System calls	No (out of scope)	Not collected; host HIDS is not the target use case.
File access	No (out of scope)	Not collected; may be added later as optional REE features, not in TA.

Scope: gateway-level IIoT/network anomaly detection, not endpoint host intrusion detection.

Q5. CPU and memory overhead — real-scenario budget analysis

*Platform: Raspberry Pi 5 (BCM2712, 4×Cortex-A76 @ 2.4 GHz, 4–8 GB DDR4). OP-TEE on Pi 5 requires patched TF-A/kernel (community port; no hardware secure RAM — TZDRAM carved from DDR) [6][7]. Reference: RPi3 OP-TEE layout reserves **16 MB TA RAM pool** + 4 MB TEE core (see OP-TEE `plat-rpi3/platform_config.h` [6]); Pi 5 port uses similar carve-out near 1 GB boundary. Cost metrics follow PICO TinyML benchmark [3] and MLPerf power guidance [13].*

Reference deployment scenario (what the budget models)

- **Benchmark / replay mode:** pre-extracted flow records from NSL-KDD (N=41) [4] or Edge-IIoTset (N≈63) [5] fed to the gateway at **5–20 scoring events/s** (not per-packet line rate).
- **Live gateway mode:** new flow summaries arrive every **1–10 s** per active flow (typical IIoT gateway aggregation [5]); target ≤20 sustained scoring events/s on one core.
- **Model class:** small int8 MLP / quantum-inspired surrogate (∼5–30 K parameters) [1][2]; **not** full MobileNet-scale vision models.
- **Important:** a 60 ms pipeline at 100 events/s would need ∼6 cores — prior "100 flows/s @ 10–15% CPU" was inconsistent; budgets below use **sustained event rate × per-event cost**.

Latency breakdown (per scoring event, ms)

Stage	Typ.	Max	Where	Basis / notes
Flow parse + feature norm	0.5	3	REE (C)	Fixed-size vector; no deep packet inspection
`TEEC_OpenSession`	0.1	1	SMC	Amortized if session kept open
World switch + `InvokeCommand`	0.5	3	REE↔TEE	OP-TEE SMC overhead [6][8]
TA inference (TinyML int8 MLP)	1	8	TEE TA	∼5–30 K MACs; fixed-point C [1]
TA inference (hybrid surrogate)	3	25	TEE TA	Larger surrogate; qubit depth capped [9][12]
REE decision + alert	0.2	2	REE	Threshold / policy lookup
End-to-end (session open)	**2**	**15**	—	TinyML-in-TA target
End-to-end (session open)	**5**	**35**	—	Hybrid-in-TA target
End-to-end (cold session)	**5**	**40**	—	Includes open/close; alert path

Service objective: $T \leq 100$ ms ingest→alert retains margin for network/cloud [5]; **not** inline 10 Gbps IPS. Latency measured via `clock_gettime` per PICO [3].

Memory map (realistic allocation)

Region	Size (target)	Justification
REE CA (C + libteec)	4–12 MB RSS	Minimal daemon; no Python on production path
REE benchmark harness (dev)	32–64 MB RSS	Python/PyTorch logging during Year 1 only
Feature + I/O buffers (REE)	∼N × 4 B + ring buffer	N=41–63 ⇒ <1 KB vector; capture buffer 1–4 MB
OP-TEE shared memory (NS)	2–4 MB	RPi3 default 4 MB SHM; holds invoke buffers
TEE core (OP-TEE OS)	∼4 MB	Code + `CFG_CORE_HEAP_SIZE` (platform)
TA RAM pool (all TAs)	∼8–16 MB	RPi3 reference; Pi 5 port similar order
`TA_STACK_SIZE`	8–16 KB	OP-TEE TA properties [6][7]; examples use 2 KB [8]
`TA_DATA_SIZE`	256–512 KB	Per-TA heap in manifest [6]; default 32 KB is **too small** for weights
Sealed model weights	10–80 KB	int8 MLP (∼5–30 K params) [1][2]; up to ∼200 KB upper bound
Working activations (TA)	4–32 KB	Fixed-point scratch; no dynamic alloc in steady state

Constraint: each TA declares `TA_DATA_SIZE`+`TA_STACK_SIZE`+binary image at load; memory is **per-TA reserved**, not shared. Early prototype merges key storage + inference + attestation into one TA to stay within pool.

CPU load vs. scoring rate (sustainability check) [1–3,6,8]

Three-pipeline comparative study; duty-cycle metric per PICO [3].

Configuration	ms/event	@ 5/s	@ 20/s	1 core duty cycle / ref.
TinyML in REE (TFLite int8)	∼2	∼1%	∼4%	Baseline [1][3]; no TEE overhead
TinyML in TA (same model)	∼8	∼4%	∼16%	+SMC [6][8]; session kept open
Hybrid surrogate in TA	∼20	∼10%	∼40%	Surrogate path [9][12]; cap size if >20/s

Formula: duty cycle ≈ (`ms/event` × events/s) / 1000 [3]. Accept criterion: TEE-wrapped TinyML ≤4× REE latency [3][6]. At 20 events/s the hybrid TA path uses ∼40% of one A76 core — acceptable for a dedicated gateway; remaining cores handle networking and OS.

Power / energy (order-of-magnitude, to be metered) [13]

Pi 5 gateway under load: ∼3.5–6 W typical (excluding attached radios). At 8 ms/event and 5 events/s: ∼40 ms CPU/s ⇒ average ∼0.3–0.5 W incremental ⇒ **∼8–15 mJ per scored flow** (USB power meter; MLPerf power methodology [13]).

Comparison rule: PyTorch→TFLite TinyML *in REE* is the accuracy/speed reference [1][2]; *TEE-wrapped* TinyML must stay within ∼2–4 times latency of REE-only [3][6]; hybrid QML must beat TinyML on recall@FPR [5][9] **within the hybrid row budget** or we report QML is not justified. **All cells above are pre-measurement targets** — the benchmark harness will log `clock_gettime` [3], `/proc/self/status` RSS [3], and TA heap usage via test commands [6][7].

References and cost-metric standards

Numbering matches inline citations on Pages 1–3.

Comparative study mapping (CPU load vs. scoring rate, Page 3)

Row 1 — TinyML in REE: TFLite Micro runtime [1], TinyML workflow [2], PICO latency/CPU/memory metrics [3].
Row 2 — TinyML in TA: same model export as Row 1 [1][2]; OP-TEE world switch + `TEE_InvokeCommand` overhead [6][7][8].
Row 3 — Hybrid surrogate in TA: offline QML training [9][10][11]; C surrogate deployability gap [12]; same TEE invoke path as Row 2 [6][8].
Datasets & detection quality: NSL-KDD [4], Edge-IIoTset [5] (recall@FPR, family hold-out for unseen IIoT attacks).
Energy reporting: MLPerf power measurement [13].

Standard cost and evaluation metrics (this study)

Metric	Definition / target	Standard / reference
Inference latency	Wall-clock ms per scoring event (session open)	PICO TinyML benchmark [3]; `clock_gettime`
End-to-end latency	Ingest → alert; target ≤100 ms	Edge IDS SLA practice [5]
CPU duty cycle	(`ms/event` × rate) / 1000 on one core	PICO CPU utilization [3]; Page 3 sustainability table
REE RSS memory	CA + buffers (MB)	`/proc/self/status`; PICO memory efficiency [3]
TA heap / stack	`TA_DATA_SIZE`, `TA_STACK_SIZE`	OP-TEE TA properties [6][7]
Model size	Sealed int8 weights (KB)	TinyML compression [1][2]
Energy / inference	Joules or mJ per scored flow	MLPerf power methodology [13]
Detection quality	Recall@FPR (e.g. 1%), AUC, F1	Edge-IIoTset IDS evaluation [5]; QML IoT prior art [9]
Trust overhead	TEE vs REE latency ratio (≤4× target)	PICO stability [3]; OP-TEE SMC + TA invoke [6][8]

Reference list

1. R. David *et al.*, "TensorFlow Lite Micro: Embedded Machine Learning on TinyML Systems," *Proc. MLSys*, 2021. *(TFLite Micro / REE inference baseline.)*
2. P. Warden and D. Situnayake, *TinyML: Machine Learning with TensorFlow Lite on Arduino and Ultra-Low-Power Microcontrollers*. O'Reilly, 2019. *(Train, quantize, deploy workflow.)*
3. A. Dey, S. Srivastava, G. Singh, and R. G. Pettit, "Real-Time Performance Benchmarking of TinyML Models in Embedded Systems (PICO)," arXiv:2509.04721, 2025. *(Latency, CPU duty cycle, memory, stability — Page 3 comparative study.)*
4. M. Tavallaee, E. Bagheri, W. Lu, and A. A. Ghorbani, "A Detailed Analysis of the KDD CUP 99 Data Set," *Proc. IEEE CISDA*, 2009. DOI: 10.1109/CISDA.2009.5356528. *(NSL-KDD dataset.)*
5. M. A. Ferrag *et al.*, "Edge-IIoTset: A New Comprehensive Realistic Cyber Security Dataset of IoT and IIoT Applications," *IEEE Access*, vol. 10, pp. 40281–40306, 2022. DOI: 10.1109/ACCESS.2022.3165809. *(Primary IIoT IDS dataset; recall@FPR evaluation.)*
6. OP-TEE Documentation, "Trusted Applications," OP-TEE v4.x. `https://optee.readthedocs.io/en/latest/building/trusted_applications.html`. *(`TA_DATA_SIZE`, `TA_STACK_SIZE`; Pi 3/5 TA pool.)*
7. OP-TEE Documentation, "Trusted Applications Architecture," OP-TEE v4.x. `https://optee.readthedocs.io/en/latest/architecture/trusted_applications.html`. *(TEE Internal Core API, `TEE_InvokeCommand`.)*
8. Linaro SWG, "TA Properties Basics," *optee_examples* docs. `https://github.com/linaro-swg/optee_examples`. *(Passthrough / inference TA; `libteec` client.)*
9. S. Chowdhury *et al.*, "Quantum Machine Learning for Anomaly Detection in IoT Devices," *Proc. AIMLSystems (Quantum Symposium)*, 2025. *(QML-for-IoT-IDS prior art; hybrid surrogate hypothesis.)*
10. V. Bergholm *et al.*, "PennyLane: Automatic differentiation of hybrid quantum-classical computation," *Advances in Quantum Technologies*, 2022. *(Hybrid training toolchain — offline only.)*
11. F. D'Amore *et al.*, "Projected Quantum Kernel for IoT Data Analysis," *Proc. IEEE PICom*, pp. 200–204, 2024. DOI: 10.1109/PICom64201.2024.00032. *(Quantum-kernel IoT classification baseline.)*
12. A. Dey and K. Raza, "Embedded Quantum Machine Learning: Feasibility and Hybrid Architectures," 2026. *(Edge QML deployability evidence gap cited in doctoral survey.)*
13. MLCommons, "MLPerf Inference Benchmark Rules," v4.x; "MLPerf Power Measurement," MLCommons, 2023–2024. `https://mlcommons.org/en/inference/`. *(Joules/inference and standardized power reporting.)*